

Assignment 16.1 Ai Assisted Coding

Htno:2303a510j7

Btno:30

Task 1: Student Information System Schema

Prompt: Design a Student Information System database schema with Students, Courses, and Enrollments tables. Define primary keys, foreign keys, and relationships. Generate SQL queries to insert a student, fetch all courses enrolled by a student, and count students in each course.

Code:

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

course_platform

my_database

Tables

products

stock_transaction

students

supplies

warehouses

Views

Stored Procedures

Functions

online_course

online_course_platform

sys

Administration Schemas

Information

No object selected

Object Info Session

SQL File 18* SQL File 19* SQL File 20* SQL File 21* SQL File 22* SQL File 23* SQL File 24* SQL File 25* x

Limit to 1000 rows

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
9	22:39:21	INSERT INTO Enrollments (student_id, course_id, enrollment_date) VALUES (1, 1, 20...	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.016 sec
10	22:39:21	SELECT * FROM Students LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
11	22:39:21	SELECT * FROM Courses LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
12	22:39:21	SELECT * FROM Enrollments LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
13	22:39:21	SELECT c.course_id, c.course_name, c.credits FROM Courses c JOIN Enrollments e ...	2 row(s) returned	0.000 sec / 0.000 sec
14	22:39:21	SELECT c.course_name, COUNT(e.student_id) AS total_students FROM Courses c L...	3 row(s) returned	0.000 sec / 0.000 sec

ENG IN 22:41 16-04-2026

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

course_platform

my_database

Tables

products

stock_transaction

students

supplies

warehouses

Views

Stored Procedures

Functions

online_course

online_course_platform

sys

Administration Schemas

Information

No object selected

Object Info Session

SQL File 18* SQL File 19* SQL File 20* SQL File 21* SQL File 22* SQL File 23* SQL File 24* SQL File 25* x

Limit to 1000 rows

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
9	22:39:21	INSERT INTO Enrollments (student_id, course_id, enrollment_date) VALUES (1, 1, 20...	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.016 sec
10	22:39:21	SELECT * FROM Students LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
11	22:39:21	SELECT * FROM Courses LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
12	22:39:21	SELECT * FROM Enrollments LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
13	22:39:21	SELECT c.course_id, c.course_name, c.credits FROM Courses c JOIN Enrollments e ...	2 row(s) returned	0.000 sec / 0.000 sec
14	22:39:21	SELECT c.course_name, COUNT(e.student_id) AS total_students FROM Courses c L...	3 row(s) returned	0.000 sec / 0.000 sec

ENG IN 22:42 16-04-2026

Output:

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

course_name	total_students
Database Management System	1
Operating Systems	1
Computer Networks	0

Explanation:

- `Students` table stores student details and `Courses` table stores course details.
- `Enrollments` table is used to connect students and courses through foreign keys.
- SQL queries are used to insert a student, fetch enrolled courses, and count students in each course.

Task 2: Hospital Management Database

Prompt: Design a Hospital Management System database with `Doctors`, `Patients`, and `Appointments` tables. Define primary keys, foreign keys, and constraints. Generate SQL queries to list all appointments for a specific doctor, retrieve patient history by patient ID, and count total patients treated by each doctor.

Code:

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

course_platform

my_database

Tables

products

stock_transaction

students

supplies

warehouses

Views

Stored Procedures

Functions

online_course

online_course_platform

sys

Administration Schemas

Information

No object selected

Object Info Session

SQL File 19* SQL File 20* SQL File 21* SQL File 22* SQL File 23* SQL File 24* SQL File 25* SQL File 26* SQL File 27*

Limit to 1000 rows

SQL File 27*

```

1 DROP DATABASE IF EXISTS hospital_management_system;
2
3 CREATE DATABASE hospital_management_system;
4 USE hospital_management_system;
5
6 CREATE TABLE Doctors (
7     doctor_id INT PRIMARY KEY AUTO_INCREMENT,
8     doctor_name VARCHAR(100) NOT NULL,
9     specialization VARCHAR(100),
10    phone VARCHAR(15) UNIQUE
11 );
12
13 CREATE TABLE Patients (
14     patient_id INT PRIMARY KEY AUTO_INCREMENT,

```

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
21	22:45:04	INSERT INTO Doctors (doctor_name, specialization, phone) VALUES ('Dr. Ramesh', 'Cardiology', '9876543210');	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.000 sec
22	22:45:04	INSERT INTO Patients (patient_name, age, gender, phone) VALUES ('Anu', 25, 'Female', '9876543210');	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.016 sec
23	22:45:04	INSERT INTO Appointments (doctor_id, patient_id, appointment_date, diagnosis) VALUES (1, 1, '2023-04-15', 'Common Cold');	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.000 sec
24	22:45:04	SELECT a.appointment_id, d.doctor_name, p.patient_name, a.appointment_date, a.diagnosis FROM Appointments a JOIN Doctors d ON a.doctor_id = d.doctor_id JOIN Patients p ON a.patient_id = p.patient_id;	2 row(s) returned	0.016 sec / 0.000 sec
25	22:45:04	SELECT a.appointment_id, a.appointment_date, a.diagnosis, d.doctor_name FROM Appointments a JOIN Doctors d ON a.doctor_id = d.doctor_id;	2 row(s) returned	0.000 sec / 0.000 sec
26	22:45:04	SELECT d.doctor_name, COUNT(DISTINCT a.patient_id) AS total_patients FROM Doctors d JOIN Appointments a ON d.doctor_id = a.doctor_id;	2 row(s) returned	0.000 sec / 0.000 sec

ENG IN 22:46 16-04-2026

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

course_platform

my_database

Tables

products

stock_transaction

students

supplies

warehouses

Views

Stored Procedures

Functions

online_course

online_course_platform

sys

Administration Schemas

Information

No object selected

Object Info Session

SQL File 19* SQL File 20* SQL File 21* SQL File 22* SQL File 23* SQL File 24* SQL File 25* SQL File 26* SQL File 27*

Limit to 1000 rows

SQL File 27*

```

19 );
20
21 CREATE TABLE Appointments (
22     appointment_id INT PRIMARY KEY AUTO_INCREMENT,
23     doctor_id INT,
24     patient_id INT,
25     appointment_date DATE,
26     diagnosis VARCHAR(255),
27     FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
28     FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
29 );
30
31 INSERT INTO Doctors (doctor_name, specialization, phone) VALUES
32 ('Dr. Ramesh', 'Cardiology', '9876543210'),

```

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

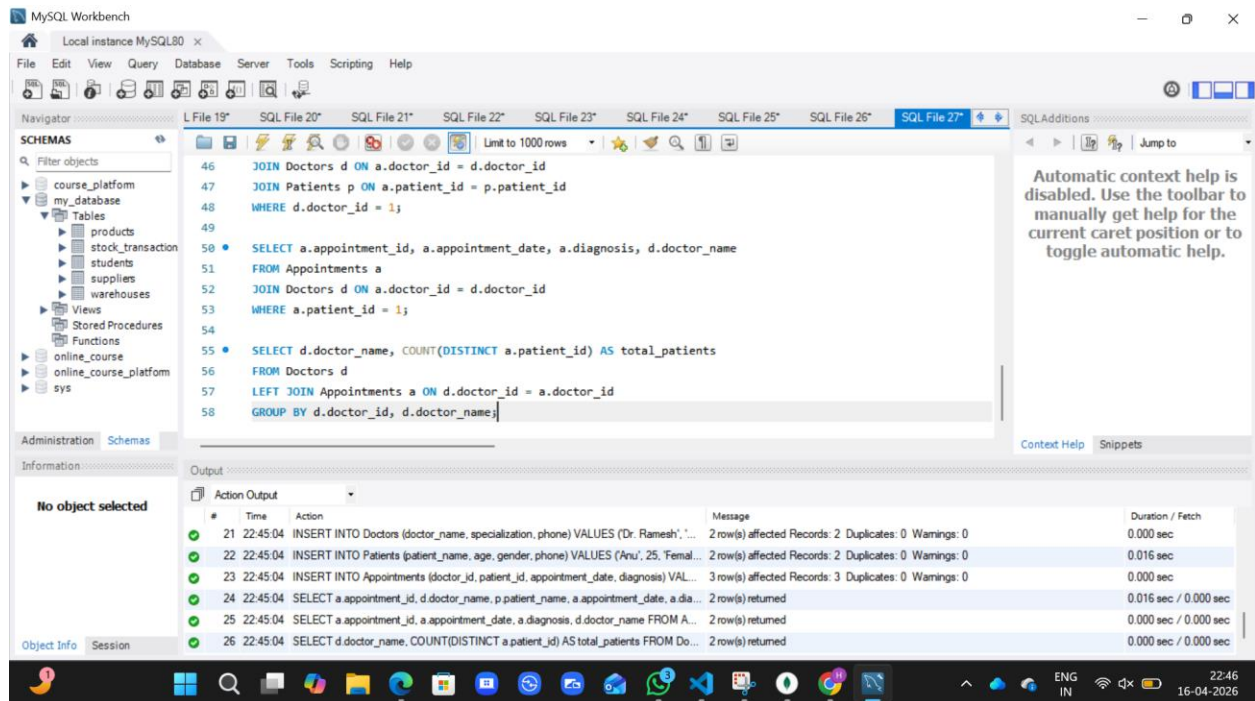
Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
21	22:45:04	INSERT INTO Doctors (doctor_name, specialization, phone) VALUES ('Dr. Ramesh', 'Cardiology', '9876543210');	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.000 sec
22	22:45:04	INSERT INTO Patients (patient_name, age, gender, phone) VALUES ('Anu', 25, 'Female', '9876543210');	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.016 sec
23	22:45:04	INSERT INTO Appointments (doctor_id, patient_id, appointment_date, diagnosis) VALUES (1, 1, '2023-04-15', 'Common Cold');	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.000 sec
24	22:45:04	SELECT a.appointment_id, d.doctor_name, p.patient_name, a.appointment_date, a.diagnosis FROM Appointments a JOIN Doctors d ON a.doctor_id = d.doctor_id JOIN Patients p ON a.patient_id = p.patient_id;	2 row(s) returned	0.016 sec / 0.000 sec
25	22:45:04	SELECT a.appointment_id, a.appointment_date, a.diagnosis, d.doctor_name FROM Appointments a JOIN Doctors d ON a.doctor_id = d.doctor_id;	2 row(s) returned	0.000 sec / 0.000 sec
26	22:45:04	SELECT d.doctor_name, COUNT(DISTINCT a.patient_id) AS total_patients FROM Doctors d JOIN Appointments a ON d.doctor_id = a.doctor_id;	2 row(s) returned	0.000 sec / 0.000 sec

ENG IN 22:46 16-04-2026



Output:

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

doctor_name	total_patients
Dr. Ramesh	2
Dr. Priya	1

Result 1

Result 2

Result 3

Output

Action Output

Explanation:

- Doctors, Patients, and Appointments are stored in separate tables to maintain proper database structure.
- Appointments table connects doctors and patients using foreign keys.
- SQL queries use JOIN and COUNT to retrieve doctor appointments, patient history, and total patients treated.

Task 3: Library Management System

Prompt: Design a Library Management System database with Books, Members, and Loans tables. Define primary keys, foreign keys, and relationships. Generate SQL queries to retrieve all

books currently issued, find overdue books where loan date is more than 30 days, and count number of books loaned by each member.

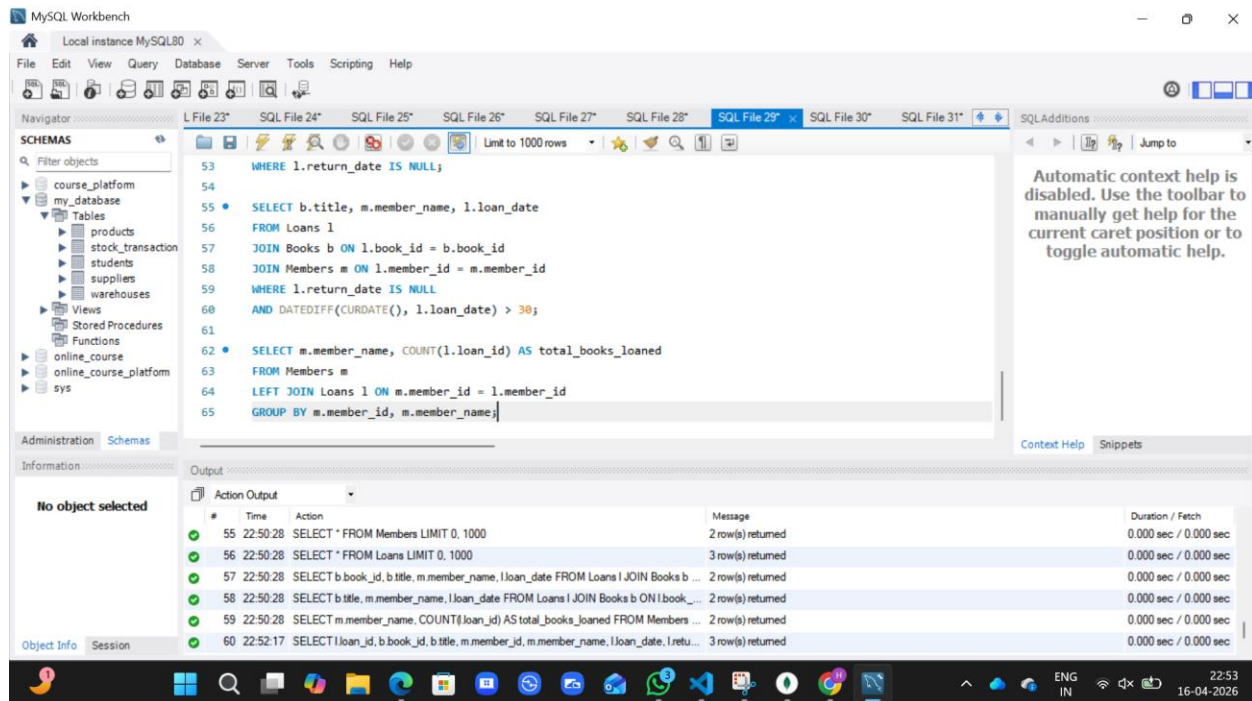
Code:

The screenshot displays the MySQL Workbench interface with a SQL script for creating a database and tables. The script is as follows:

```
1 DROP DATABASE IF EXISTS library_management_system;
2
3 CREATE DATABASE library_management_system;
4 USE library_management_system;
5
6 CREATE TABLE Books (
7     book_id INT PRIMARY KEY AUTO_INCREMENT,
8     title VARCHAR(150) NOT NULL,
9     author VARCHAR(100),
10    category VARCHAR(50),
11    available_copies INT NOT NULL
12 );
13
14 CREATE TABLE Members (
15     member_id INT PRIMARY KEY AUTO_INCREMENT,
16     member_name VARCHAR(100) NOT NULL,
17     address VARCHAR(200),
18     phone VARCHAR(20),
19     email VARCHAR(100)
20 );
21
22 CREATE TABLE Loans (
23     loan_id INT PRIMARY KEY AUTO_INCREMENT,
24     book_id INT NOT NULL,
25     member_id INT NOT NULL,
26     loan_date DATE NOT NULL,
27     return_date DATE,
28     FOREIGN KEY (book_id) REFERENCES Books(book_id),
29     FOREIGN KEY (member_id) REFERENCES Members(member_id)
30 );
31
32 CREATE INDEX idx_loans_book_id ON Loans(book_id);
33 CREATE INDEX idx_loans_member_id ON Loans(member_id);
```

The Output window shows the execution results of the script:

#	Time	Action	Message	Duration / Fetch
55	22:50:28	SELECT * FROM Members LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
56	22:50:28	SELECT * FROM Loans LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
57	22:50:28	SELECT b.book_id, b.title, m.member_name, l.loan_date FROM Loans l JOIN Books b ON l.book_id = b.book_id	2 row(s) returned	0.000 sec / 0.000 sec
58	22:50:28	SELECT b.title, m.member_name, l.loan_date FROM Loans l JOIN Books b ON l.book_id = b.book_id	2 row(s) returned	0.000 sec / 0.000 sec
59	22:50:28	SELECT m.member_name, COUNT(l.loan_id) AS total_books_loaned FROM Members m JOIN Loans l ON m.member_id = l.member_id	2 row(s) returned	0.000 sec / 0.000 sec
60	22:52:17	SELECT l.loan_id, b.book_id, b.title, m.member_id, m.member_name, l.loan_date, l.return_date FROM Loans l JOIN Books b ON l.book_id = b.book_id JOIN Members m ON l.member_id = m.member_id	3 row(s) returned	0.000 sec / 0.000 sec



Output:

loan_id	book_id	title	member_id	member_name	loan_date	return_date
1	1	Database System Concepts	1	Hasini	2026-03-01	NULL
3	3	Computer Networks	1	Hasini	2026-04-01	2026-04-10
2	2	Operating System Principles	2	Kavya	2026-02-15	NULL

Explanation:

1. Books, Members, and Loans are stored in separate tables for proper normalization.
2. Loans table connects members and books using foreign keys.
3. Queries are used to retrieve issued books, find overdue books, and count books borrowed by each member.

Task 4: Online Shopping Database

Prompt: Design an Online Shopping Database for an E-commerce platform with Users, Products, Orders, and OrderDetails tables. Define primary keys, foreign keys, and relationships. Generate SQL queries to retrieve all orders by a user, find the most purchased product, and calculate total revenue in a given month. Show inserted values and detailed output with IDs.

Code:

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

Filter objects

SCHEMAS

- hospital_management_system
- my_database
 - products
 - stock_transactions
 - students
 - supplies
 - warehouses
 - Views
 - Stored Procedures
 - Functions
- online_course
- online_course_platform
- student_information_system
- online_shopping_database
- sys

Administration Schemas

Information

Schema: online_shopping_database

Object Info Session

SQL File 27* SQL File 28* SQL File 29* SQL File 30* SQL File 31* SQL File 32* SQL File 33* SQL File 34*

Limit to 1000 rows

```

1 DROP DATABASE IF EXISTS online_shopping_database;
2
3 CREATE DATABASE online_shopping_database;
4 USE online_shopping_database;
5
6 DROP TABLE IF EXISTS OrderDetails;
7 DROP TABLE IF EXISTS Orders;
8 DROP TABLE IF EXISTS Products;
9 DROP TABLE IF EXISTS Users;
10
11 CREATE TABLE Users (
12     user_id INT PRIMARY KEY AUTO_INCREMENT,
13     user_name VARCHAR(100) NOT NULL,
14     email VARCHAR(100) UNIQUE NOT NULL,

```

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
79	22:55:45	SELECT * FROM OrderDetails LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
80	22:55:45	SELECT o.order_id, u.user_id, u.user_name, o.order_date FROM Orders o JOIN	2 row(s) returned	0.000 sec / 0.000 sec
81	22:55:45	SELECT od.order_detail_id, o.order_id, u.user_name, p.product_id, p.product_	4 row(s) returned	0.000 sec / 0.000 sec
82	22:55:45	SELECT p.product_id, p.product_name, SUM(od.quantity) AS total_purchased ...	1 row(s) returned	0.000 sec / 0.000 sec
83	22:55:45	SELECT MONTH(o.order_date) AS order_month, YEAR(o.order_date) AS order...	1 row(s) returned	0.000 sec / 0.000 sec
84	22:57:33	SELECT od.order_detail_id, o.order_id, u.user_id, u.user_name, p.product_id, p...	4 row(s) returned	0.000 sec / 0.000 sec

ENG IN 22:58 16-04-2026

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

Filter objects

SCHEMAS

- hospital_management_system
- my_database
 - products
 - stock_transactions
 - students
 - supplies
 - warehouses
 - Views
 - Stored Procedures
 - Functions
- online_course
- online_course_platform
- student_information_system
- online_shopping_database
- sys

Administration Schemas

Information

Schema: online_shopping_database

Object Info Session

SQL File 27* SQL File 28* SQL File 29* SQL File 30* SQL File 31* SQL File 32* SQL File 33* SQL File 34*

Limit to 1000 rows

```

41
42 INSERT INTO Users (user_name, email, address) VALUES
43 ('Hasini', 'hasini_shop@gmail.com', 'Hyderabad'),
44 ('Teja', 'teja_shop@gmail.com', 'Warangal');
45
46 INSERT INTO Products (product_name, price, stock) VALUES
47 ('Laptop', 50000.00, 10),
48 ('Mouse', 500.00, 50),
49 ('Keyboard', 1200.00, 30);
50
51 INSERT INTO Orders (user_id, order_date) VALUES
52 (1, '2026-04-01'),
53 (1, '2026-04-05'),
54 (2, '2026-04-06');

```

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

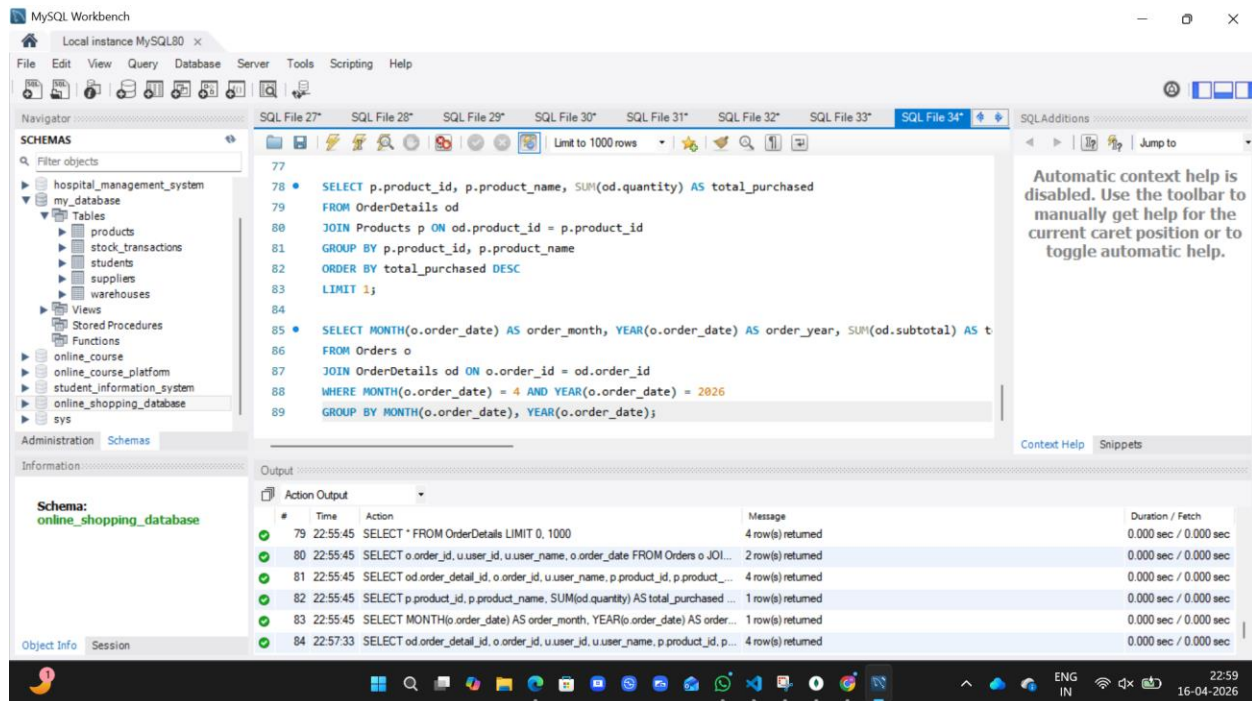
Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
79	22:55:45	SELECT * FROM OrderDetails LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
80	22:55:45	SELECT o.order_id, u.user_id, u.user_name, o.order_date FROM Orders o JOIN	2 row(s) returned	0.000 sec / 0.000 sec
81	22:55:45	SELECT od.order_detail_id, o.order_id, u.user_name, p.product_id, p.product_	4 row(s) returned	0.000 sec / 0.000 sec
82	22:55:45	SELECT p.product_id, p.product_name, SUM(od.quantity) AS total_purchased ...	1 row(s) returned	0.000 sec / 0.000 sec
83	22:55:45	SELECT MONTH(o.order_date) AS order_month, YEAR(o.order_date) AS order...	1 row(s) returned	0.000 sec / 0.000 sec
84	22:57:33	SELECT od.order_detail_id, o.order_id, u.user_id, u.user_name, p.product_id, p...	4 row(s) returned	0.000 sec / 0.000 sec

ENG IN 22:58 16-04-2026



Output:

order_detail_id	order_id	user_id	user_name	product_id	product_name	quantity	subtotal	order_date
1	1	1	Hasini	1	Laptop	1	50000.00	2026-04-01
2	1	1	Hasini	2	Mouse	2	1000.00	2026-04-01
3	2	1	Hasini	3	Keyboard	1	1200.00	2026-04-05
4	3	2	Teja	2	Mouse	3	1500.00	2026-04-06

Explanation:

- Users, Products, Orders, and OrderDetails are stored in separate tables to maintain normalization.
- Orders table stores user order details, and OrderDetails table stores product-wise order information.
- SQL queries are used to display inserted values, retrieve user orders, identify the most purchased product, and calculate monthly revenue.

Task 5: University Examination Database

Prompt: Design a University Examination System database with Students, Subjects, Exams, and Results tables. Define primary keys, foreign keys, and referential integrity constraints. Generate SQL queries to insert examination results for a student, retrieve subject-wise marks for a student, and calculate average marks for each subject. Show inserted values and detailed output with IDs.

Code:

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- hospital_management_system
- my_database
 - products
 - stock_transactions
 - students
 - supplies
 - warehouses
- Views
- Stored Procedures
- Functions
- online_course
- online_course_platform
- student_information_system
- online_shopping_database
- sys

Administration Schemas

Information

Schema: online_shopping_database

Object Info Session

SQL File 29* SQL File 30* SQL File 31* SQL File 32* SQL File 33* SQL File 34* SQL File 35* SQL File 36*

Limit to 1000 rows

```

1 DROP DATABASE IF EXISTS university_examination_system;
2
3 CREATE DATABASE university_examination_system;
4 USE university_examination_system;
5
6 DROP TABLE IF EXISTS Results;
7 DROP TABLE IF EXISTS Exams;
8 DROP TABLE IF EXISTS Subjects;
9 DROP TABLE IF EXISTS Students;
10
11 CREATE TABLE Students (
12     student_id INT PRIMARY KEY AUTO_INCREMENT,
13     student_name VARCHAR(100) NOT NULL,
14     department VARCHAR(50),

```

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
101	23:01:01	SELECT * FROM Subjects LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
102	23:01:01	SELECT * FROM Exams LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
103	23:01:01	SELECT * FROM Results LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
104	23:01:01	SELECT r.result_id, s.student_id, s.student_name, sub.subject_id, sub.subject_name, r.result_id, s.student_id, s.student_name, sub.subject_id, sub.subject_name, r.ma...	4 row(s) returned	0.000 sec / 0.000 sec
105	23:01:01	SELECT s.student_id, s.student_name, sub.subject_id, sub.subject_name, r.ma...	2 row(s) returned	0.000 sec / 0.000 sec
106	23:01:01	SELECT sub.subject_id, sub.subject_name, AVG(r.marks) AS average_marks F...	3 row(s) returned	0.016 sec / 0.000 sec

23:01 16-04-2026

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- hospital_management_system
- my_database
 - products
 - stock_transactions
 - students
 - supplies
 - warehouses
- Views
- Stored Procedures
- Functions
- online_course
- online_course_platform
- student_information_system
- online_shopping_database
- sys

Administration Schemas

Information

Schema: online_shopping_database

Object Info Session

SQL File 29* SQL File 30* SQL File 31* SQL File 32* SQL File 33* SQL File 34* SQL File 35* SQL File 36*

Limit to 1000 rows

```

22 );
23
24 CREATE TABLE Exams (
25     exam_id INT PRIMARY KEY AUTO_INCREMENT,
26     subject_id INT NOT NULL,
27     exam_name VARCHAR(100) NOT NULL,
28     exam_date DATE NOT NULL,
29     FOREIGN KEY (subject_id) REFERENCES Subjects(subject_id)
30 );
31
32 CREATE TABLE Results (
33     result_id INT PRIMARY KEY AUTO_INCREMENT,
34     student_id INT NOT NULL,
35     exam_id INT NOT NULL,

```

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
101	23:01:01	SELECT * FROM Subjects LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
102	23:01:01	SELECT * FROM Exams LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
103	23:01:01	SELECT * FROM Results LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
104	23:01:01	SELECT r.result_id, s.student_id, s.student_name, sub.subject_id, sub.subject_name, r.result_id, s.student_id, s.student_name, sub.subject_id, sub.subject_name, r.ma...	4 row(s) returned	0.000 sec / 0.000 sec
105	23:01:01	SELECT s.student_id, s.student_name, sub.subject_id, sub.subject_name, r.ma...	2 row(s) returned	0.000 sec / 0.000 sec
106	23:01:01	SELECT sub.subject_id, sub.subject_name, AVG(r.marks) AS average_marks F...	3 row(s) returned	0.016 sec / 0.000 sec

23:01 16-04-2026

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

Filter objects

SCHEMAS

- hospital_management_system
- my_database
 - Tables
 - products
 - stock_transactions
 - students
 - supplies
 - warehouses
 - Views
 - Stored Procedures
 - Functions
- online_course
- online_course_platform
- student_information_system
- online_shopping_database
- sys

Administration Schemas

Information

Schema: online_shopping_database

Object Info Session

SQL File 29* SQL File 30* SQL File 31* SQL File 32* SQL File 33* SQL File 34* SQL File 35* SQL File 36*

Limit to 1000 rows

```

72 SELECT s.student_id, s.student_name, sub.subject_id, sub.subject_name, r.marks, r.grade
73 FROM Results r
74 JOIN Students s ON r.student_id = s.student_id
75 JOIN Exams e ON r.exam_id = e.exam_id
76 JOIN Subjects sub ON e.subject_id = sub.subject_id
77 WHERE s.student_id = 1;
78
79
80 SELECT sub.subject_id, sub.subject_name, AVG(r.marks) AS average_marks
81 FROM Results r
82 JOIN Exams e ON r.exam_id = e.exam_id
83 JOIN Subjects sub ON e.subject_id = sub.subject_id
84 GROUP BY sub.subject_id, sub.subject_name;

```

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
101	23:01:01	SELECT * FROM Subjects LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
102	23:01:01	SELECT * FROM Exams LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
103	23:01:01	SELECT * FROM Results LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
104	23:01:01	SELECT r.result_id, s.student_id, s.student_name, sub.subject_id, sub.subject_name, r.marks, r.grade	4 row(s) returned	0.000 sec / 0.000 sec
105	23:01:01	SELECT s.student_id, s.student_name, sub.subject_id, sub.subject_name, r.marks, r.grade	2 row(s) returned	0.000 sec / 0.000 sec
106	23:01:01	SELECT sub.subject_id, sub.subject_name, AVG(r.marks) AS average_marks	3 row(s) returned	0.016 sec / 0.000 sec

23:01 16-04-2026

Output:

Result Grid

Filter Rows:

Export: Wrap Cell Content:

	subject_id	subject_name	average_marks
▶	1	DBMS	85.250000
	2	Operating Systems	76.000000
	3	Computer Networks	69.500000

Explanation:

- Students, Subjects, Exams, and Results are stored in separate tables for a normalized database design.
- Results table connects students and exams, while Exams table connects subjects.
- SQL queries are used to show inserted values, retrieve subject-wise student marks, and calculate average marks for each subject.